claude-windows / 7dc3da51-af38-47b6-83f6-fdda83b60664

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\subagents\workflows\wf_c5395511-084\agent-aa2ae5712e3746e60.jsonl`
- SHA-256: `ec0e22a80804bd5cb2c5da377636b3903dd22f61ca2df376947276fb298bc634`
- Source modified: `2026-07-06T21:33:53+00:00`
- Imported at: `2026-07-08T16:00:10+00:00`
- project: `wf_c5395511-084`
- session_id: `7dc3da51-af38-47b6-83f6-fdda83b60664`

Transcript

1. user (2026-07-06T21:29:35.076Z)

```
Reviewing GitHub PR #51 "feat(policy): add F2 topic preferences" (branch feat/f2-topic-
preferences -> main). ADR 0012 F2.
+417/-16, 8 files. A checked-out copy is at C:/Users/avidu/Projects/_wt-pr51 (read files there,
or 'git -C C:/Users/avidu/Projects/Telegram show origin/feat/f2-topic-preferences:<path>').
Run repros: PYTHONPATH=C:/Users/avidu/Projects/_wt-pr51/src
C:/Users/avidu/Projects/Telegram/.venv/Scripts/python.exe <script>  (db._apply_schema on
aiosqlite ":memory:").

WHAT F2 DOES:
- models.py: VideoRecord gains a 'text' field (caption). db.py: insert_video /
_content_item_to_video /
  get_video_by_content_fingerprint / list_videos_for_user propagate text into/out of
content_items.text (which existed since ADR 0011).
- policy.py: TopicPreferences + parse_topic_preferences + load_topic_preferences
(json.loads(policy.config_json)); TopicPreferenceStage
  (Tier 2): exclude-veto, include any/all, casefold substring match over source.title +
content_item.text.
- telethon_client.py: _message_text(message) (message/raw_text, strip, None if empty);
_content_item_preview(...) builds a throwaway
  ContentItem (incl. a computed dedup_key) for topic eval; _evaluate_topic_preferences(conn,
account, source, item) loads the default
  policy and runs TopicPreferenceStage. on_new_message (and backfill.py) now:
matched=engine.evaluate(); if matched and source:
  run topic stage, matched=topic_result.passed; then insert VideoRecord(text=...) and deliver if
matched.
- The live handler on_new_message and backfill_user are '# pragma: no cover'.

VERIFIED by the main reviewer already: caption text reaches content_items.text; topics survive a
/set (F1's merge) and are read via
the RAW config (not FilterConfig); topic gating passes all-includes / vetoes excludes. Gate:
ruff clean, 513 passed @ 95.58%.
Your job: find bugs/regressions/edge cases in the WIRING and stage the happy path missed.

Ground rules: READ code, cite file:line. Run repros. Report ONLY defects in THIS PR's diff. No
praise. Concrete failure scenario.
Rank most-severe first. Severity: high (real bug/regression/wrong delivery decision), medium
(edge/latent/design-inconsistency that will bite), low (hygiene/scope-note).

ADVERSARIALLY VERIFY this finding ? try to REFUTE it. Read cited code, run a repro with
PYTHONPATH=C:/Users/avidu/Projects/_wt-pr51/src
C:/Users/avidu/Projects/Telegram/.venv/Scripts/python.exe.
CONFIRMED only if the defect genuinely ships in THIS PR's diff and produces the described wrong
behaviour. REFUTED if handled / can't occur /
```

pre-existing / out-of-scope-and-disclosed. PLAUSIBLE if unsettled. Corrected severity.
FINDING (design_tests): New topic-wiring helpers _content_item_preview and
_evaluate_topic_preferences are entirely untested (happy path missed; coverage regressed
96.63->95.58)
severity medium | src/tg_video_filter/telethon_client.py:385
summary: The two functions that translate a live message into a policy-evaluable ContentItem and
run the TopicPreferenceStage have zero test coverage, so a wiring regression (wrong field
mapping, wrong account_id, source-None handling) would ship green.
scenario: If a future refactor mis-maps text/title into the ContentItem or passes the wrong id
to get_default_policy, the unit tests for TopicPreferenceStage (which are fed hand-built
contexts) still pass and CI stays green, so a matched video is silently dropped or an excluded
one delivered in production. Coverage confirms lines 395-396 (_content_item_preview) and 430-433
(_evaluate_topic_preferences) are never executed by the suite; grep of tests/ finds no reference
to either symbol though both are in __all__.
evidence: pytest --cov term-missing reports telethon_client.py missing 395-396, 430-433;
policy.py missing 61,83,98,110,140,150; `grep _content_item_preview|_evaluate_topic_preferences
tests/` -> no files. Yet a repro (seed user+channel+source, save_policy_config topics include=AI
exclude=Crypto, call _content_item_preview then _evaluate_topic_preferences) exercises them
cleanly: include-via-title -> passed, Crypto caption -> vetoed, no-policy -> None. These helpers
are NOT behind pragma:no-cover, unlike on_new_message/backfill_user, so they are directly unit-
testable.
rec: Add a unit test that seeds a source + default policy with topics and asserts
_evaluate_topic_preferences returns pass on include-match, fail on exclude-veto, and None when
no default policy exists; assert _content_item_preview copies text/title/document_id into the
ContentItem. This restores coverage and locks the wiring contract the pragma'd handlers depend
on.

2. assistant (2026-07-06T21:29:37.233Z)

I'll verify this finding by examining the code and checking whether these helpers are actually
untested.

3. user (2026-07-06T21:29:39.247Z)

```
370             return getattr(media, "id", None)
371         return None
372
373
374     def _message_text(message: object) -> str | None:
375         """Return caption/message text normalized for storage, or None if empty."""
376         text = getattr(message, "message", None)
377         if text is None:
378             text = getattr(message, "raw_text", None)
379         if not isinstance(text, str):
380             return None
381         text = text.strip()
382         return text or None
383
384
385     def _content_item_preview(
386         account_id: int,
387         source: Source,
388         message_id: int,
389         document_id: int | None,
390         meta: VideoMeta,
391         text: str | None,
392         matched: bool,
393     ) -> ContentItem:
394         """Build the ContentItem shape used by policy stages before insert."""
395         assert source.id is not None
396         return ContentItem(
397             account_id=account_id,
398             source_id=source.id,
399             provider=source.provider,
400             provider_message_id=str(message_id),
```

```
            content_type="video",
            text=text,
            document_id=str(document_id) if document_id is not None else None,
            duration_s=meta.duration_s,
            width=meta.width,
            height=meta.height,
            size_bytes=meta.size_bytes,
            mime_type=meta.mime_type,
            dedup_key=db.compute_dedup_key(
                provider=source.provider,
                document_id=str(document_id) if document_id is not None else None,
                source_id=source.id,
                provider_message_id=str(message_id),
                size_bytes=meta.size_bytes,
                duration_s=meta.duration_s,
                height=meta.height,
                width=meta.width,
            ),
            matched=matched,
        )


    async def _evaluate_topic_preferences(
        conn: aiosqlite.Connection,
        account_id: int,
        source: Source,
        content_item: ContentItem,
    ) -> StageResult | None:
        """Evaluate ADR 0012 topic preferences for a candidate item."""
        policy = await db.get_default_policy(conn, account_id)
        if policy is None:
            return None
        return await TopicPreferenceStage().evaluate(
            PolicyContext(content_item=content_item, source=source, policy=policy)
        )


    def _peer_id(peer: object) -> int:
        """Best-effort numeric id for the peer of a message.

        Telegram represents channel/group/user peers as different TL types; we
        collapse them to a single int for dedup/storage. The exact subclass isn't
        important ? only that the id is stable per chat.
        """
        # Each Peer* type has a field like channel_id / chat_id / user_id.
        for field in ("channel_id", "chat_id", "user_id"):
            val = getattr(peer, field, None)
            if val is not None:
                return int(val)
```

4. user (2026-07-06T21:29:40.960Z)

```
C:/Users/avidu/Projects/_wt-pr51/tests/test_telethon_extraction.py:28:    _message_text,
C:/Users/avidu/Projects/_wt-pr51/tests/test_telethon_extraction.py:128:def
test_message_text_strips_caption() -> None:
C:/Users/avidu/Projects/_wt-pr51/tests/test_telethon_extraction.py:131:    assert
_message_text(msg) == "AI philosophy clip"
C:/Users/avidu/Projects/_wt-pr51/tests/test_telethon_extraction.py:134:def
test_message_text_empty_returns_none() -> None:
C:/Users/avidu/Projects/_wt-pr51/tests/test_telethon_extraction.py:137:    assert
_message_text(msg) is None
C:/Users/avidu/Projects/_wt-pr51/tests/test_telethon_extraction.py:140:def
test_message_text_raw_text_fallback() -> None:
C:/Users/avidu/Projects/_wt-pr51/tests/test_telethon_extraction.py:144:    assert
_message_text(Msg()) == "Bangalore explorations"
```

```
Binary file C:/Users/avidu/Projects/_wt-
pr51/tests/__pycache__/test_telethon_extraction.cpython-313-pytest-8.4.2.pyc matches
```

5. assistant (2026-07-06T21:29:43.780Z)

```
The finding claims `_content_item_preview` and `_evaluate_topic_preferences` are entirely
untested. Let me verify by checking coverage on the actual PR diff and confirming the grep
results.
```

6. user (2026-07-06T21:29:46.563Z)

```
---EXIT---
```

7. user (2026-07-06T21:29:47.740Z)

```
119:) -> TelegramClient:  # pragma: no cover
176:    async def on_new_message(event: NewMessage.Event) -> None:  # pragma: no cover
275:            topic_result = await _evaluate_topic_preferences(
279:                _content_item_preview(
385:def _content_item_preview(
423:async def _evaluate_topic_preferences(
454:def _channel_title(event: NewMessage.Event) -> str | None:  # pragma: no cover
466:__all__ = [
470:    "_content_item_preview",
471:    "_evaluate_topic_preferences",
```

8. assistant (2026-07-06T21:29:51.034Z)

```
Confirmed: no test references either helper. Both are in `__all__` (lines 470-471) and are NOT
behind `# pragma: no cover` (the pragmas are on lines 119, 176, 454 ? the client factory and
live handlers). Now let me run coverage to confirm the lines are missing and verify the helpers
are directly testable.
```

9. user (2026-07-06T21:30:04.472Z)

```
src\tg_video_filter\telethon_client.py           79     10     32      5    85%   78, 95->108,
353, 368, 380, 395-396, 430-433, 463->exit
513 passed in 9.19s
```

10. assistant (2026-07-06T21:30:07.780Z)

```
Coverage confirms the exact missing lines cited in the finding: `395-396`
(`_content_item_preview` body: the `assert` and `return ContentItem(...)`) and `430-433`
(`_evaluate_topic_preferences` body). 513 passed.

Now let me verify the helpers are actually directly testable (not blocked by some import or
fixture issue) with a repro.
```

11. user (2026-07-06T21:30:10.006Z)

```
203:async def _apply_schema(conn: aiosqlite.Connection) -> None:
266:def compute_dedup_key(
1284:async def get_default_policy(conn: aiosqlite.Connection, account_id: int) -> Policy | None:
1326:async def save_policy_config(
```

12. user (2026-07-06T21:30:10.178Z)

```
1       """Telethon ingestion: client factory, event handler, and metadata extraction.
2
3       Architecture (see docs/design.md §5):
4
5           NewMessage event
6             -> extract_video_meta(message)   # pure, unit-tested
```

```
7             -> dedup via db.get_video()
8             -> FilterEngine.evaluate(meta)
9             -> db.insert_video(...) + delivery.forward_to_target(...,
target=cfg.delivery_target)
10
11      The metadata extraction is deliberately a **pure** function taking a Telethon
12      ``Message`` and returning a ``VideoMeta | None``. This isolates the only piece
13      of logic that's worth unit-testing (everything else is Telethon/async glue).
14      """
15
16      from __future__ import annotations
17
18      import logging
19      import sqlite3
20      from pathlib import Path
21      from typing import Protocol
22
23      import aiosqlite
24      from telethon import TelegramClient
25      from telethon.events import NewMessage
26      from telethon.tl.types import (
27          Document,
28          DocumentAttributeAnimated,
29          DocumentAttributeVideo,
30          Message,
31          MessageMediaDocument,
32      )
33
34      from tg_video_filter import db
35      from tg_video_filter.commands import ENGINE_KEY
36      from tg_video_filter.delivery import build_link_text, deliver
37      from tg_video_filter.filters import FilterEngine, VideoMeta
38      from tg_video_filter.models import (
39          Channel,
40          ContentItem,
41          DeliveryMode,
42          Source,
43          TelegramSession,
44          User,
45          VideoRecord,
46      )
47      from tg_video_filter.policy import PolicyContext, StageResult, TopicPreferenceStage
48
49      log = logging.getLogger(__name__)
50
```

13. assistant (2026-07-06T21:30:12.598Z)

Let me write a repro that seeds a source + default policy and calls both helpers, confirming they are directly unit-testable and run cleanly.

14. user (2026-07-06T21:30:15.048Z)

```
1326:async def save_policy_config(
async def save_policy_config(
    conn: aiosqlite.Connection, account_id: int, config: dict[str, object]
) -> None:
    """Persist the raw default policy config object."""
    existing = await get_default_policy(conn, account_id)
    await upsert_policy(
        conn,
        Policy(
            id=existing.id if existing is not None else None,
            account_id=account_id,
            name=existing.name if existing is not None else "default",
```

```
            config_json=_policy_config_dict_to_json(config),
            is_default=True,
            created_at=(existing.created_at if existing is not None else _now()),
            updated_at=_now(),
        ),
    )
```

--- Destination / RoutingRule / DeliveryAttempt repos (ADR 0011, Track C) --

```
async def upsert_destination(conn: aiosqlite.Connection, dest: Destination) -> Destination:
    """Insert or update a destination row by (account, provider, target)."""
    await conn.execute(
        """INSERT INTO destinations
             (account_id, provider, target, kind, title, is_output, created_at)
           VALUES (?, ?, ?, ?, ?, ?, ?)
           ON CONFLICT(account_id, provider, target) DO UPDATE SET
             kind=excluded.kind,
             title=excluded.title,
             is_output=excluded.is_output""",
        (
            dest.account_id,
            dest.provider,
```

15. user (2026-07-06T21:30:20.490Z)

```
C:/Users/avidu/Projects/_wt-pr51/src/tg_video_filter/db.py:1108:async def upsert_source(conn:
aiosqlite.Connection, source: Source) -> Source:
C:/Users/avidu/Projects/_wt-pr51/src/tg_video_filter/models.py:123:class Source(BaseModel):
```

16. assistant (2026-07-06T21:30:24.101Z)

```
I have enough to build the repro. Let me construct Source and VideoMeta from their definitions.
```

17. user (2026-07-06T21:30:25.601Z)

```
123     class Source(BaseModel):
124         """A configured place we ingest content from (ADR 0011).
125
126         Generalizes the legacy :class:`Channel` (1:1 for Telegram today; future
127         providers add their own source kinds). ``provider_source_id`` is the
128         provider-native id (Telegram ``channel_id``, future WhatsApp group id, ?).
129         """
130
131         model_config = ConfigDict(frozen=True)
132
133         id: int | None = Field(default=None, description="Surrogate PK (DB-assigned)")
134         account_id: int = Field(description="FK ? accounts.id (owner of the source)")
135         provider: str = Field(default="telegram", description="Provider namespace")
136         provider_source_id: str = Field(
137             description="Provider-native source id (Telegram channel_id, ?)"
138         )
139         title: str | None = None
140         kind: str = Field(default="channel", description="'channel' | 'group' | 'feed'")
141         watched: bool = Field(
142             default=False,
143             description="ADR 0009 selective scanning: ingest from this source iff True",
144         )
145         seen_first_at: datetime = Field(default_factory=_utcnow)
146         last_seen_at: datetime | None = None
147
148
149     class ContentItem(BaseModel):
```

```
150         """A normalized content record discovered from any :class:`Source` (ADR 0011).
151
152         Generalizes the legacy :class:`VideoRecord`. Phase 1 stores only videos
```

18. user (2026-07-06T21:30:26.947Z)

```
grep: C:/Users/avidu/Projects/_wt-pr51/src/tg_video_filter/filters.py: No such file or directory
```

19. user (2026-07-06T21:30:32.013Z)

```
C:/Users/avidu/Projects/_wt-pr51/src/tg_video_filter/filters/base.py:15:class
VideoMeta(BaseModel):
---
15:class VideoMeta(BaseModel):
16-    """Metadata extracted from a Telegram video message.
17-
18-    All fields are optional because Telegram metadata can be partial; filters
19-    decide how to treat ``None`` (by default: treat as a non-match with a
20-    descriptive reason).
21-    """
22-
23-    duration_s: int | None = None
24-    width: int | None = None
25-    height: int | None = None
26-    size_bytes: int | None = None
27-    mime_type: str | None = None
28-    is_gif: bool | None = None
29-    is_round: bool | None = None
30-    nosound: bool | None = None
```

20. assistant (2026-07-06T21:30:34.673Z)

Now I have everything. Let me write the repro.

21. assistant (2026-07-06T21:30:46.510Z)

```
import asyncio
import aiosqlite
from tg_video_filter import db
from tg_video_filter.models import Source
from tg_video_filter.filters.base import VideoMeta
from tg_video_filter.telethon_client import (
    _content_item_preview,
    _evaluate_topic_preferences,
)


async def main():
    conn = await aiosqlite.connect(":memory:")
    conn.row_factory = aiosqlite.Row
    await db._apply_schema(conn)

    account_id = 1
    # seed a source
    src = Source(account_id=account_id, provider_source_id="chan123",
                 title="AI Weekly Roundup", watched=True)
    src = await db.upsert_source(conn, src)
    await conn.commit()

    meta = VideoMeta(duration_s=30, width=1280, height=720,
                     size_bytes=1000, mime_type="video/mp4")

    # 1) _content_item_preview field mapping
    item = _content_item_preview(
```

```
        account_id=account_id, source=src, message_id=42,
        document_id=999, meta=meta, text="a caption about Crypto", matched=True,
    )
    assert item.text == "a caption about Crypto", item.text
    assert item.document_id == "999", item.document_id
    assert item.account_id == account_id
    assert item.source_id == src.id
    assert item.dedup_key is not None
    print("preview OK: text/doc/account/source/dedup mapped")

    # 2) no default policy -> None
    r = await _evaluate_topic_preferences(conn, account_id, src, item)
    print("no-policy result:", r)
    assert r is None

    # 3) include=AI (matches title), exclude=Crypto (matches caption -> veto)
    await db.save_policy_config(conn, account_id, {
        "topics": {"include_any": ["AI"], "exclude": ["Crypto"]}
    })
    await conn.commit()
    r2 = await _evaluate_topic_preferences(conn, account_id, src, item)
    print("with-policy (crypto caption) passed:", r2.passed, "reason:", r2.reason)

    # 4) clean caption, include matches title
    item2 = _content_item_preview(
        account_id=account_id, source=src, message_id=43,
        document_id=None, meta=meta, text="clean video", matched=True,
    )
    r3 = await _evaluate_topic_preferences(conn, account_id, src, item2)
    print("with-policy (clean caption) passed:", r3.passed, "reason:", r3.reason)

    await conn.close()
    print("ALL HELPERS EXECUTED CLEANLY -> directly unit-testable")


asyncio.run(main())
```

22. user (2026-07-06T21:30:46.511Z)

```
<tool_use_error>File has not been read yet. Read it first before writing to it.</tool_use_error>
```

23. user (2026-07-06T21:32:58.737Z)

```
Exit code 143
Command timed out after 2m 0s
Traceback (most recent call last):
  File "C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\scratchpad\repro.py", line 59, in <module>
    asyncio.run(main())
    ~~~~~~~~~~~^^^^^^^^
  File "C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\asyncio\runners.py", line
195, in run
    return runner.run(main)
           ~~~~~~~~~~^^^^^^
  File "C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\asyncio\runners.py", line
118, in run
    return self._loop.run_until_complete(task)
           ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~^^^^^^
  File "C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\asyncio\base_events.py", line
725, in run_until_complete
    return future.result()
           ~~~~~~~~~~~~~^^
  File "C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\scratchpad\repro.py", line 20, in main
    src = await db.upsert_source(conn, src)
```

```
         ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "C:\Users\avidu\Projects\_wt-pr51\src\tg_video_filter\db.py", line 1124, in upsert_source
    await conn.execute(
    ...<17 lines>...
    )
  File "C:\Users\avidu\Projects\Telegram\.venv\Lib\site-packages\aiosqlite\core.py", line 223,
in execute
    cursor = await self._execute(self._conn.execute, sql, parameters)
             ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "C:\Users\avidu\Projects\Telegram\.venv\Lib\site-packages\aiosqlite\core.py", line 160,
in _execute
    return await future
           ^^^^^^^^^^^^
  File "C:\Users\avidu\Projects\Telegram\.venv\Lib\site-packages\aiosqlite\core.py", line 63, in
_connection_worker_thread
    result = function()
sqlite3.IntegrityError: FOREIGN KEY constraint failed
```

24. assistant (2026-07-06T21:33:03.212Z)

The FK constraint failed because I need to seed an account first. But this doesn't affect the finding ? the helpers themselves are already proven testable (the coverage output shows they're the only untested lines, and `_content_item_preview` is a pure function). The FK issue is just my repro's missing account seed, and the timeout was the connection hanging on the exception. Let me fix the seed to fully exercise `_evaluate_topic_preferences` too.

25. user (2026-07-06T21:33:05.804Z)

```
397:        INSERT INTO accounts (id, telegram_id, provider, created_at)
940:async def upsert_account(conn: aiosqlite.Connection, account: Account) -> Account:
943:        """INSERT INTO accounts (id, telegram_id, provider, display_name, created_at)
---schema accounts---
42:CREATE TABLE IF NOT EXISTS accounts (
51:    account_id    INTEGER PRIMARY KEY REFERENCES accounts(id),
61:    user_id       INTEGER NOT NULL REFERENCES accounts(id),
70:    user_id     INTEGER NOT NULL REFERENCES accounts(id),
86:    user_id     INTEGER PRIMARY KEY REFERENCES accounts(id),
98:    account_id         INTEGER NOT NULL REFERENCES accounts(id),
111:    account_id           INTEGER NOT NULL REFERENCES accounts(id),
134:    account_id  INTEGER NOT NULL REFERENCES accounts(id),
149:    account_id  INTEGER NOT NULL REFERENCES accounts(id),
161:    account_id     INTEGER NOT NULL REFERENCES accounts(id),
```

26. user (2026-07-06T21:33:10.212Z)

```
35:class Account(BaseModel):
36-    """A product identity ? the owner of config, channels, routes, and content.
37-
38-    Decoupled from any specific provider's session (ADR 0010). ``telegram_id``
39-    is the bridge used by the Bot API front-door (``from.id``) and by the
40-    legacy userbot path; it is nullable so future providers that don't have a
41-    Telegram id (e.g. WhatsApp) can still own an ``Account``.
42-    """
43-
44-    model_config = ConfigDict(frozen=True)
45-
46-    id: int = Field(description="Stable surrogate PK; the owner key for all downstream
tables")
47-    telegram_id: int | None = Field(
```

27. user (2026-07-06T21:33:25.380Z)

```
preview OK: text/doc/account/source/dedup mapped
no-policy result: None
```

```
crypto-caption passed: False | reason: excluded topic matched: Crypto
clean-caption passed: True | reason: no include topics configured
ALL HELPERS EXECUTED CLEANLY -> directly unit-testable
```

28. assistant (2026-07-06T21:33:37.929Z)

The repro confirms both helpers are directly unit-testable and behave correctly: preview maps text/document_id/account/source/dedup_key; `_evaluate_topic_preferences` returns None with no policy, vetoes the Crypto caption (passed=False), and passes the clean caption. Yet no test in the suite exercises either symbol.

Now let me assess severity. The finding is classified as `design_tests` / medium. This is a coverage-gap / test-hygiene finding, not a live bug ? the code is correct today. Let me quickly confirm the coverage-regression claim aspect (96.63?95.58) is a project-wide floor concern versus these lines specifically, but the core defect (untested wiring helpers not behind pragma) is proven.

Assessment:
- The two helpers are genuinely untested (grep: zero references; coverage: lines 395-396, 430-433 missing).
- They are in `__all__` (exported, line-item public API) and NOT behind `# pragma: no cover`, unlike the live handlers ? so they are meant to be testable and were simply skipped.
- The repro proves they are trivially unit-testable and the "happy path" is missed.
- This is a real test-coverage gap in THIS PR's diff (these helpers are new in this PR). But it is not a shipping runtime bug ? the wiring is correct today. It's a latent-risk / hygiene finding: severity medium is defensible but leans toward the low end since there's no wrong behavior today, only unprotected wiring. Given the project's CLAUDE.md coverage-floor discipline and that these are the load-bearing translation from live message ? policy eval that the pragma'd handlers depend on, medium is appropriate.

29. user (2026-07-06T21:33:52.884Z)

Structured output provided successfully